

Lecture-16 [22 September]

Tying Loose Ends : Proof of Theorem 5.1

Theorem 5.1 : Fix any $\pi = \{\pi_n\}_{n \geq 0} \in \Pi^{\text{HR}}$. For each $s \in \mathcal{S}$, there exists

$$\pi'_s = \{\pi'_{n,s}\}_{n \geq 0} \in \Pi^{\text{MR}} \text{ such that}$$

$$\underbrace{\sum_{n=1}^{\infty} \lambda^{n-1} E^{\pi} \left[x(S_{n-1}, A_{n-1}) \mid S_0 = s \right]}_{V_{\omega, \lambda}^{\pi}(s)} = \underbrace{\sum_{n=1}^{\infty} \lambda^{n-1} E^{\pi'_s} \left[x(S_{n-1}, A_{n-1}) \mid S_0 = s \right]}_{V_{\omega, \lambda}^{\pi'_s}(s)}.$$

The proof of the above result is based on the following lemma.

Lemma 5.9 :

Let $\pi = \{\pi_n\}_{n \geq 0} \in \Pi^{\text{HR}}$. For each $s \in \mathcal{S}$, there exists $\pi'_s = \{\pi'_{n,s}\}_{n \geq 0} \in \Pi^{\text{MR}}$ such that

$$\begin{aligned} P^{\pi} \left(S_{n+1} = s'', S_n = s', A_n = a \mid S_0 = s \right) \\ = P^{\pi'_s} \left(S_{n+1} = s'', S_n = s', A_n = a \mid S_0 = s \right) \end{aligned}$$

holds for all $s', s'' \in \mathcal{S}$, $a \in \mathcal{A}$.

Proof of Lemma 5.9 :

Fix an arbitrary $s \in \mathcal{S}$. Construct $\pi'_s = \{\pi'_{0,s}, \pi'_{1,s}, \pi'_{2,s}, \dots\}$ as

$\pi'_{n,s} : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ for each $n \geq 0$, where

$$\begin{aligned} \pi'_{n,s}(a|s') &= P^{\pi}(A_n = a \mid S_n = s', S_0 = s) \\ &= \frac{P^{\pi}(S_n = s', A_n = a \mid S_0 = s)}{P^{\pi}(S_n = s' \mid S_0 = s)}, \quad s' \in \mathcal{S}, a \in \mathcal{A}. \end{aligned}$$

Clearly, $\pi'_s \in \Pi^{\text{MR}}$ by definition. We then have

$$\begin{aligned} &P^{\pi}(S_{n+1} = s'', S_n = s', A_n = a \mid S_0 = s) \\ &= P^{\pi}(S_n = s' \mid S_0 = s) \cdot P^{\pi}(A_n = a \mid S_n = s', S_0 = s) \cdot P^{\pi}(S_{n+1} = s'' \mid S_n = s', A_n = a, S_0 = s). \end{aligned}$$

Due to the MDP structure,

$$\begin{aligned}
 P^\pi \left(S_{n+1} = s'' \mid S_n = s', A_n = a, S_0 = s \right) &= P^\pi \left(S_{n+1} = s'' \mid S_n = s', A_n = a \right) \\
 &= P \left(s'' \mid s', a \right) \\
 &= P^{\pi_s} \left(S_{n+1} = s'' \mid S_n = s', A_n = a \right) \\
 &= P^{\pi_s} \left(S_{n+1} = s'' \mid S_n = s', A_n = a, S_0 = s \right)
 \end{aligned}$$

Also, by construction, it follows that

$$\begin{aligned}
 P^\pi \left(A_n = a \mid S_n = s', S_0 = s \right) &= \pi'_{n,s} (a \mid s') \\
 &= P^{\pi_s} \left(A_n = a \mid S_n = s' \right) \\
 &= P^{\pi_s} \left(A_n = a \mid S_n = s', S_0 = s \right) \quad \forall n \geq 0.
 \end{aligned}$$

We now claim that

$$P^\pi \left(S_n = s' \mid S_0 = s \right) = P^{\pi_s} \left(S_n = s' \mid S_0 = s \right) \quad \forall s' \in \mathcal{S}, n \geq 0.$$

The claim is trivially true for $n=0$. Assuming that the claim holds for $t=1, \dots, n-1$, we have

$$\begin{aligned}
 P^\pi \left(S_n = s' \mid S_0 = s \right) &= \sum_{l \in \mathcal{S}} \sum_{a \in \mathcal{A}} P^\pi \left(S_{n-1} = l, A_{n-1} = a, S_n = s' \mid S_0 = s \right) \\
 &= \sum_{l \in \mathcal{S}} \sum_{a \in \mathcal{A}} P^\pi \left(S_{n-1} = l \mid S_0 = s \right) \cdot P^\pi \left(A_{n-1} = a \mid S_{n-1} = l, S_0 = s \right) \\
 &\quad \cdot P^\pi \left(S_n = s' \mid S_{n-1} = l, A_{n-1} = a, S_0 = s \right) \\
 &= \sum_{l \in \mathcal{S}} \sum_{a \in \mathcal{A}} \underbrace{P^{\pi_s} \left(S_{n-1} = l \mid S_0 = s \right)}_{\text{comes from induction hypothesis}} \cdot P^{\pi_s} \left(A_{n-1} = a \mid S_{n-1} = l, S_0 = s \right) \\
 &\quad \cdot P \left(s' \mid l, a \right) \\
 &= \sum_{l \in \mathcal{S}} \sum_{a \in \mathcal{A}} P^{\pi_s} \left(S_{n-1} = l \mid S_0 = s \right) \cdot P^{\pi_s} \left(A_{n-1} = a \mid S_{n-1} = l, S_0 = s \right) \\
 &\quad P^{\pi_s} \left(S_n = s' \mid S_{n-1} = l, A_{n-1} = a, S_0 = s \right) \\
 &= P^{\pi_s} \left(S_n = s' \mid S_0 = s \right).
 \end{aligned}$$

The policy $\pi'_S = \{\pi'_{n,S}\}_{n \geq 0}$ constructed in the proof of Lemma 5.9 satisfies Theorem 5.1: for each $s \in \mathcal{S}$,

$$\begin{aligned} V_{\infty, \lambda}^{\pi}(s) &= \sum_{n=1}^{\infty} \lambda^{n-1} E^{\pi} [r(S_{n-1}, A_{n-1}) \mid S_0 = s] \\ &= \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{s' \in \mathcal{S}} \sum_{a \in \mathcal{A}} P^{\pi}(S_{n-1} = s', A_{n-1} = a \mid S_0 = s) r(s', a) \\ &= \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{s' \in \mathcal{S}} \sum_{a \in \mathcal{A}} P^{\pi'_S}(S_{n-1} = s', A_{n-1} = a \mid S_0 = s) r(s', a) \\ &= V_{\infty, \lambda}^{\pi'_S}(s). \end{aligned}$$

Finally, let $\pi' = \{\pi'_n\}_{n \geq 0}$ be defined as

$$\pi' = \sum_{s \in \mathcal{S}} \pi'_s 1_{\{S_0 = s\}}.$$

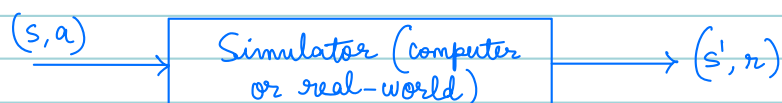
Clearly, $\pi' \in \Pi^{\text{MR}}$, and

$$V_{\infty, \lambda}^{\pi}(s) = V_{\infty, \lambda}^{\pi'}(s) \quad \forall s \in \mathcal{S}.$$

Simulation in Tabular Models

[Chapter 10, Puterman and Chan]

$$\begin{aligned} V_{\infty, \lambda}^{\phi} &= r_{\phi} + \lambda P_{\phi} V_{\infty, \lambda}^{\phi} \\ &\downarrow \\ r_{\phi}(s) &= \sum_a \phi(a|s) r(s, a) \\ &\text{OR} \\ &r(s, \phi(s)) \end{aligned}$$



A **simulator**, be it computer-based or a real-time process, generates a state-reward pair (s', r) from the current state-action pair (s, a) . Simulation may be:

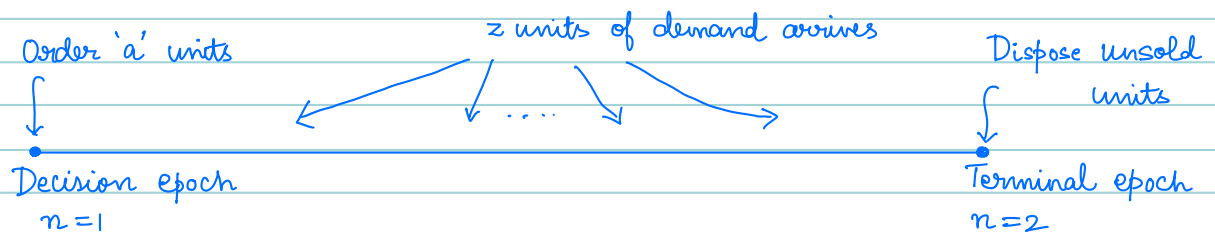
- **Model-based**: Here, $\mathcal{S}$, $\mathcal{A}$, P and r are known, and $s' \sim P(\cdot | s, a)$
- **Model-free**: Here, $\mathcal{A}$ is assumed to be known, while $\mathcal{S}$, P , r may not be known. A real-world process generates

$s' \sim P(\cdot | s, a)$ according to the underlying MDP's TPM P .

We will broadly study the following simulation methods:

- Monte-Carlo
- Temporal differencing
- Q-learning and SARSA.

Understanding the above methods through an example:
The Newsvendor Problem



- At the start of the period, a newsvendor purchases 'a' units of product at a cost of 'c' per unit (newspapers, magazines, etc)
- Thereafter, a random demand of 'z' units arrives throughout the period
- Each item is sold by the newsvendor at a price of $g > c$ per unit
- At the end of the period, each unsold item is disposed of at a scrap value of $h < c$

Goal of newsvendor: Pick an order quantity a^* that maximizes the one-period expected reward.

MDP formulation:

$S_0 = 0$, $A_0 \in \{0, \dots, a_{\max}\}$, $S_1 \in \{0, \dots, a_{\max}\}$
 ↪ initial inventory ↪ units purchased at start ↪ # unsold units = ending inventory
 (Note: $a_{\max}$ is the maximum # newspapers/magazines that the newsvendor can purchase)

Denoting the random demand by Z , we have

$$P(s' | 0, a) = \begin{cases} P(Z = a - s') , & s' = 1, \dots, a, \\ P(Z \geq a) , & s' = 0, \\ 0 , & s' > a. \end{cases}$$

The revenue accrued by the newsvendor in one period is

$$\text{Revenue}(a, z) = \begin{cases} (g-c)z + (h-c)(a-z), & z < a, \\ (g-c)a, & z \geq a. \end{cases}$$

$$= \begin{cases} Gz - L(a-z), & z < a, \\ Ga, & z \geq a. \end{cases} \quad \begin{matrix} G := g-c \\ L := c-h \end{matrix}$$

$$= G \min\{z, a\} - L(a-z)_+ \quad x_+ = \max\{x, 0\}.$$

Thus, the expected revenue at purchase quantity of a units is

$$\begin{aligned} q(a) &:= E[\text{Revenue}(a, Z)] \\ &= \sum_{z=0}^{\infty} \text{Revenue}(a, z) \cdot P(Z=z) \\ &= \sum_{z=0}^{a-1} [Gz - L(a-z)] P(Z=z) + Ga P(Z \geq a). \end{aligned}$$

Central optimisation problem: find

$$a^* \in \arg \max_{a \in \{0, \dots, a_{\max}\}} q(a).$$

Remark: If the distribution of Z is known, then

$$a^* = F_Z^{-1} \left(\frac{G}{G+L} \right), \quad \left(\text{rounded appropriately to make } a^* \text{ an integer} \right)$$

where F_Z is the CDF of Z , and

$$F_Z^{-1}(u) := \inf \{z \in \mathbb{R} : F_Z(z) \geq u\}, \quad u \in (0, 1).$$

a) Monte-Carlo Simulation:

- Obtain IID samples $z_1, z_2, \dots, z_N$ from the underlying distribution F_Z

- Set

$$\hat{q}_N(a) := \frac{1}{N} \sum_{t=1}^N \text{Revenue}(a, z_t), \quad a \in \{0, \dots, a_{\max}\}.$$

- Find

$$a_N^* \in \arg \max_a \hat{q}_N(a).$$

Remarks:

- The same sequence $z_1, \dots, z_N$ could be used to evaluate $\hat{q}_N(a)$ for all a .
- The choice of a particular ' a ' is to be viewed as a policy. Then,
$$\hat{q}_N(a) = \frac{1}{N} \sum_{t=1}^N \text{Revenue}(a, z_t)$$

is policy value function corresponding to policy ' a '.

- Defining $r(s, a, s')$ as

$$r(o, a, s') = \begin{cases} -ga + c(a - s') + hs' & , \quad s' \leq a, \\ 0 & , \quad s' > a, \end{cases}$$

One can show that $E^{\pi} [r(o, a, s_t)] = q(a)$, where $\pi \equiv \{a\}$.
 $\downarrow \quad \downarrow$
 $S_0 \quad A_0$

b) Temporal differencing:

- Start with $q_1(a) = 0 \quad \forall a$.
- Use the update

$$\begin{aligned} \forall a, \forall n \geq 1: \quad q_{n+1}(a) &= (1 - \tau_n) q_n(a) + \tau_n \text{Revenue}(a, z_n) \\ &= q_n(a) + \tau_n (\text{Revenue}(a, z_n) - q_n(a)), \end{aligned}$$

Where $\{\tau_n\}_{n \geq 1}$ is a sequence of positive constants converging to 0.

τ_n is the learning rate at iteration n .

Remarks:

- ① $\{\tau_n\}_{n \geq 1}$ is usually chosen so that $\sum_{n=1}^{\infty} \tau_n^2 < +\infty$, $\sum_{n=1}^{\infty} \tau_n = +\infty$.
(Eg: $\tau_n = 1/n$)
- ② The above TD approach is referred to as TD(0).

c) Q-learning and SARSA

Here, we use the recursion

Q-learning: $q_{n+1}(a) = q_n(a) + \tau_n \left(\max_{a'} \text{Revenue}(a', z_n) - q_n(a) \right), \quad n \geq 1.$

Alternatively, one may sample $a' \sim \text{Unif}\{0, \dots, a_{\max}\}$ and use

SARSA: $q_{n+1}(a) = q_n(a) + \tau_n (\text{Revenue}(a', z_n) - q_n(a)), \quad n \geq 1.$

* A mix of both strategies would be :

$$\text{Pick } a' = \left\{ \begin{array}{ll} a^* \in \arg \max_a \text{Revenue}(a, z_n) & , \text{ w.p. } 1 - \epsilon_n, \\ a \in \{0, \dots, a_{\max}\} \setminus \{a^*\} & , \text{ w.p. } \frac{\epsilon_n}{a_{\max}}, \end{array} \right\} \begin{array}{l} \epsilon_n\text{-greedy} \\ \text{Sampling} \end{array}$$

and set

$$q_{n+1}(a) = q_n(a) + \tau_n \left(\text{Revenue}(a', z_n) - q_n(a) \right).$$

* Another alternative to sample a' is using softmax rule

$$p_n(a') = \frac{e^{\eta_n q_n(a')}}{\sum_a e^{\eta_n q_n(a)}},$$

and setting

$$q_{n+1}(a) = q_n(a) + \tau_n \left(\text{Revenue}(a', z_n) - q_n(a) \right).$$

Q-Learning : An "Earning While Learning" Perspective

- Set $q_1(a) = 0 \quad \forall a, \quad R_0 = 0$.
- Choose a_1 greedily, ϵ -greedily, or by softmax sampling
- Implement a_1 and observe $r_1 = \text{Revenue}(a_1, z_1)$
- Set $R_1 = R_0 + r_1$
- Update $q_2(a_1)$ using one of Q-learning or SARSA update rules
- $\vdots$
- Repeat for a total of N iterations
- At the end, we have R_N and $q_N = (q_N(a))_a$.

By tuning η_n, τ_n and based on update rule, different variants of Q-learning can be compared on the basis of running average of cumulative reward & on how accurately they determine a^* .